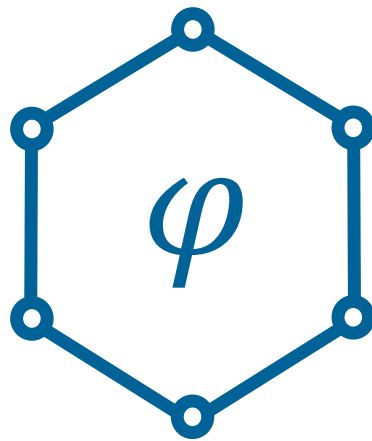


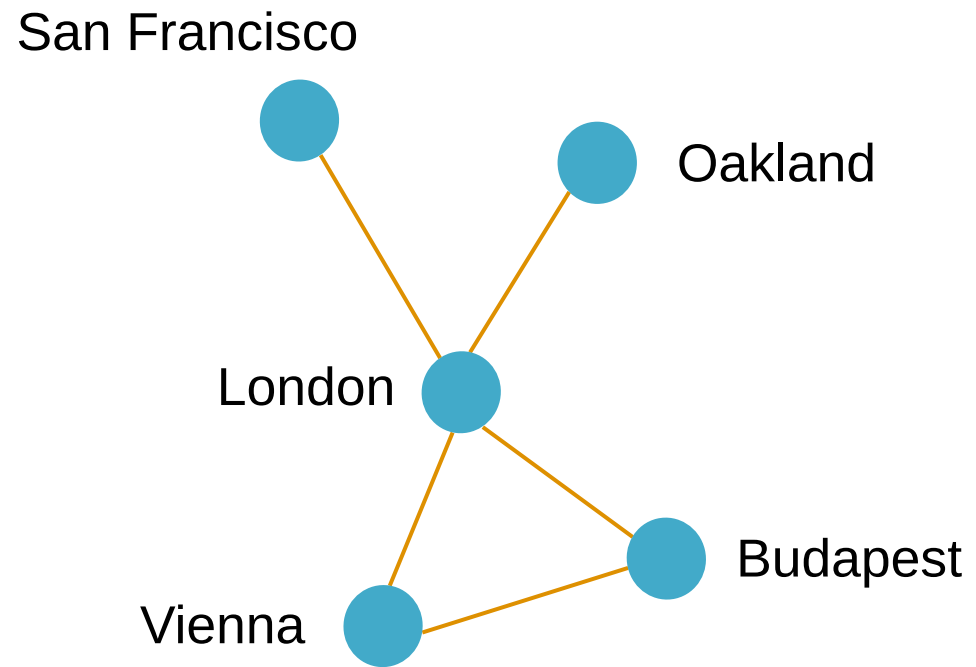


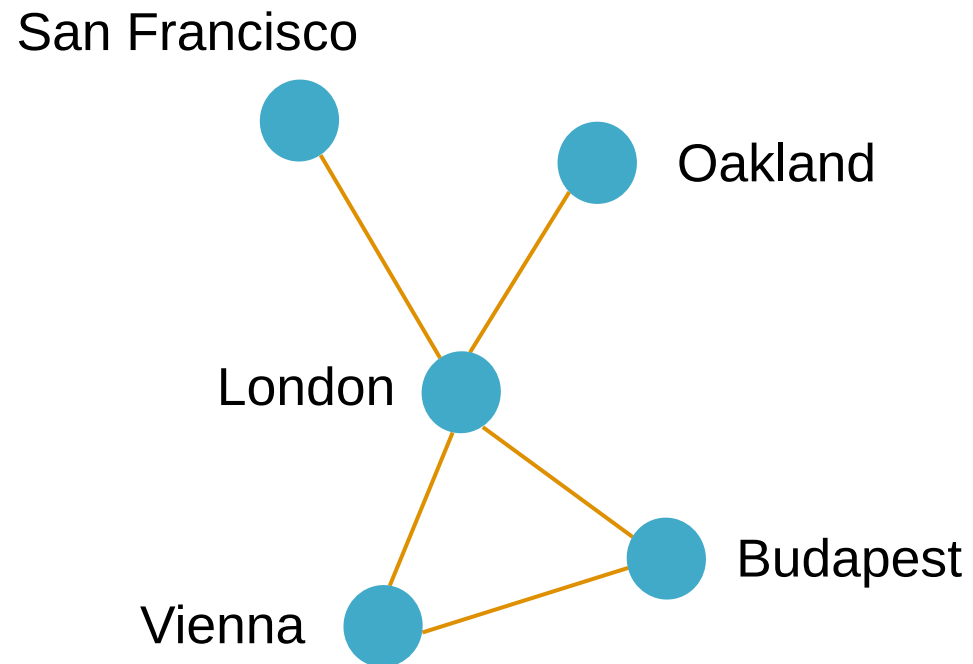
Logical Knowledge in KGs

Recursion in SQL

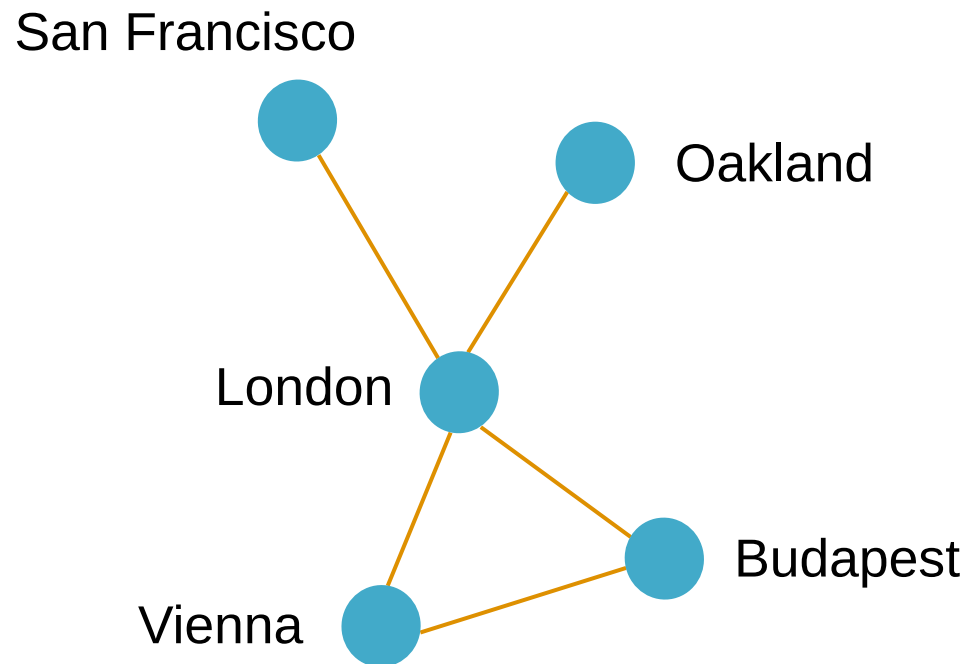
Primer

Emanuel Sallinger





airport	
IATA	City
VIE	Vienna
LHR	London
LGW	London
BUD	Budapest
OAK	Oakland
SFO	San Francisco



flight

Origin	Destination	Airline
VIE	LHR	BA
LHR	SFO	BA
LGW	OAK	DI
BUD	VIE	OS

airport

IATA	City
VIE	Vienna
LHR	London
LGW	London
BUD	Budapest
OAK	Oakland
SFO	San Francisco

airport

IATA	City
VIE	Vienna
LHR	London
SFO	San Francisco



city

City
Vienna
London
San Francisco

airport

IATA	City
VIE	Vienna
LHR	London
SFO	San Francisco

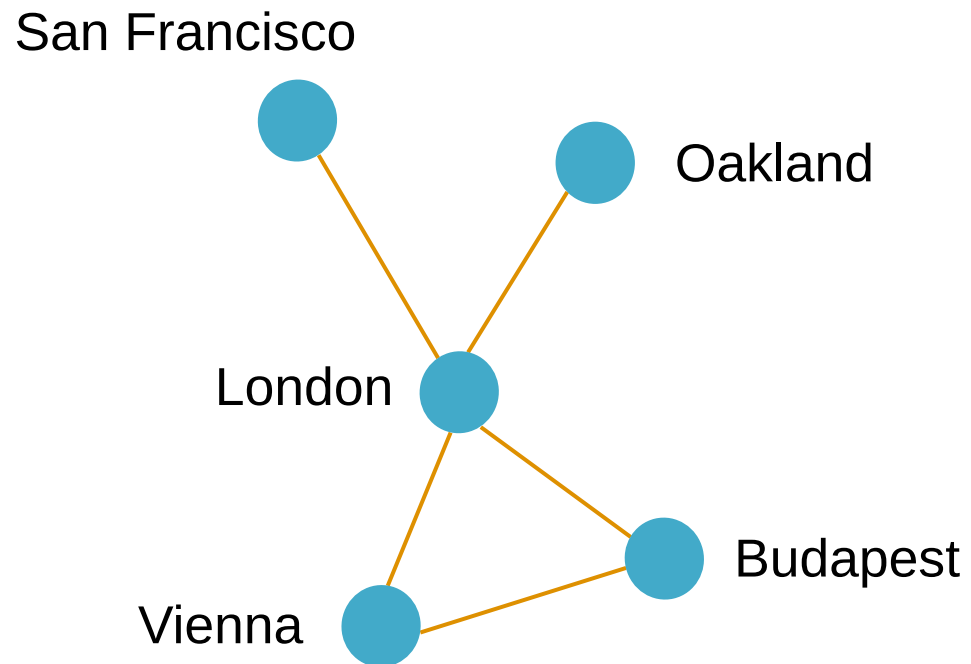


city

City
Vienna
London
San Francisco

SQL

```
SELECT City INTO city FROM airport
```

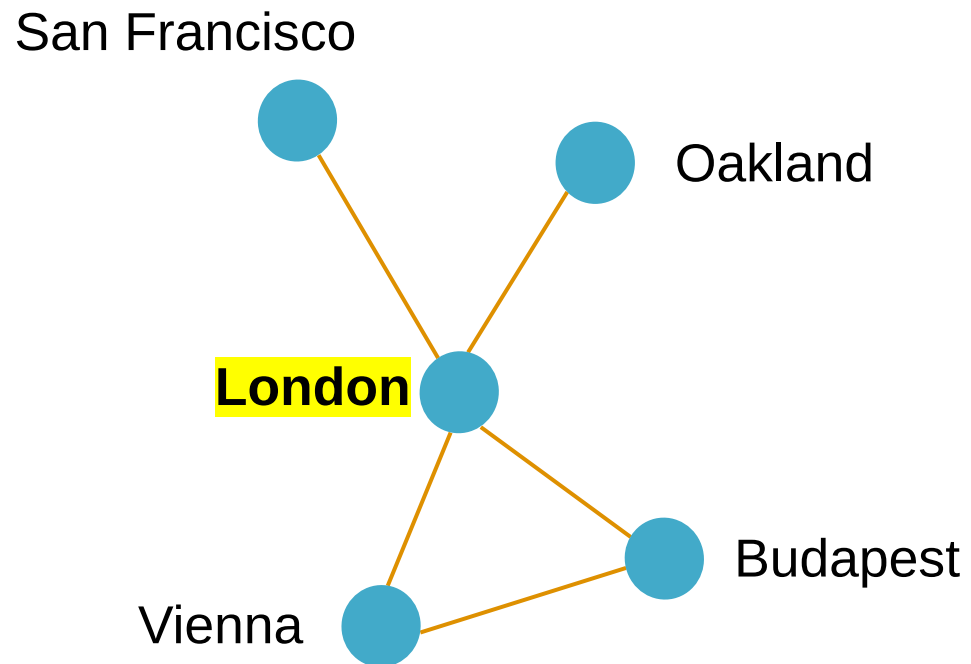


flight

Origin	Destination	Airline
VIE	LHR	BA
LHR	SFO	BA
LGW	OAK	DI
BUD	VIE	OS

airport

IATA	City
VIE	Vienna
LHR	London
LGW	London
BUD	Budapest
OAK	Oakland
SFO	San Francisco

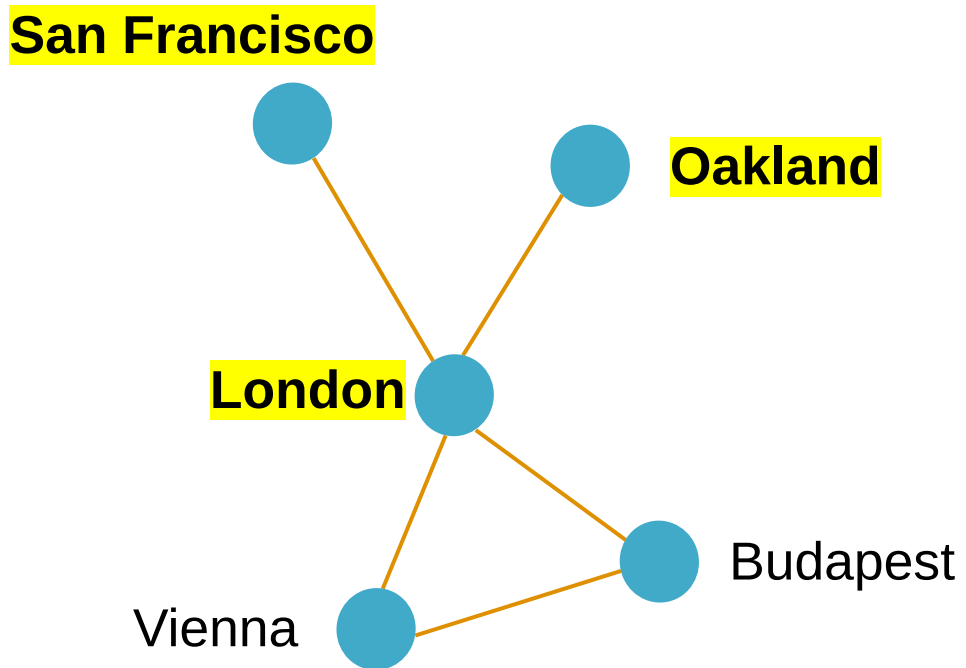


flight

Origin	Destination	Airline
VIE	LHR	BA
LHR	SFO	BA
LGW	OAK	DI
BUD	VIE	OS

airport

IATA	City
VIE	Vienna
LHR	London
LGW	London
BUD	Budapest
OAK	Oakland
SFO	San Francisco



flight

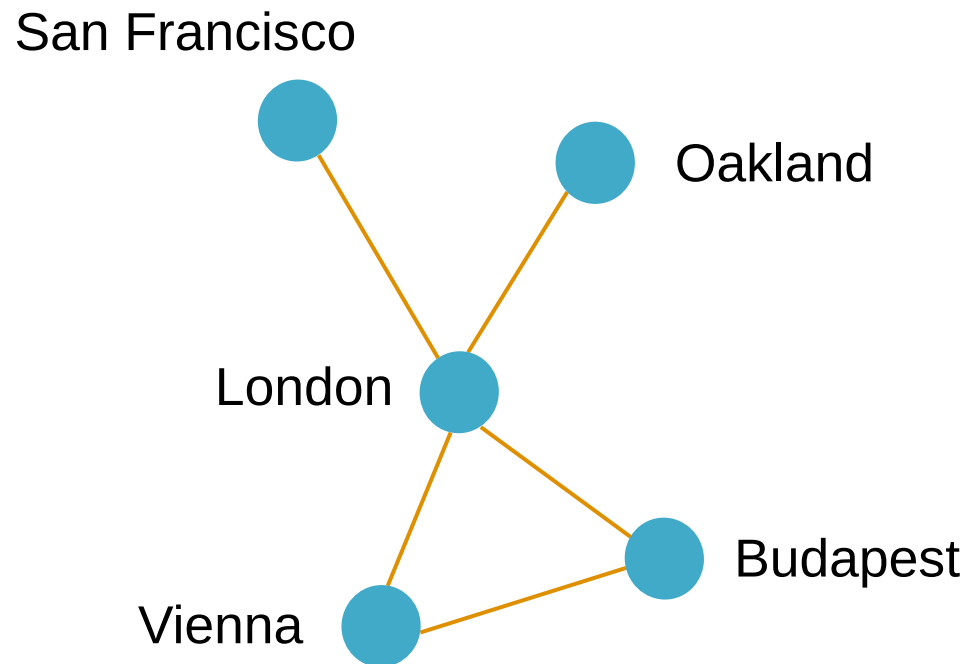
Origin	Destination	Airline
VIE	LHR	BA
LHR	SFO	BA
LGW	OAK	DI
BUD	VIE	OS

airport

IATA	City
VIE	Vienna
LHR	London
LGW	London
BUD	Budapest
OAK	Oakland
SFO	San Francisco

```
SELECT f.Destination INTO dest  
FROM flight f, airport a  
WHERE f.Origin = a.IATA AND a.City = "London"
```

```
SELECT f.Destination INTO dest  
FROM flight f  
INNER JOIN airport a  
ON f.Origin = a.IATA  
WHERE a.City = "London"
```

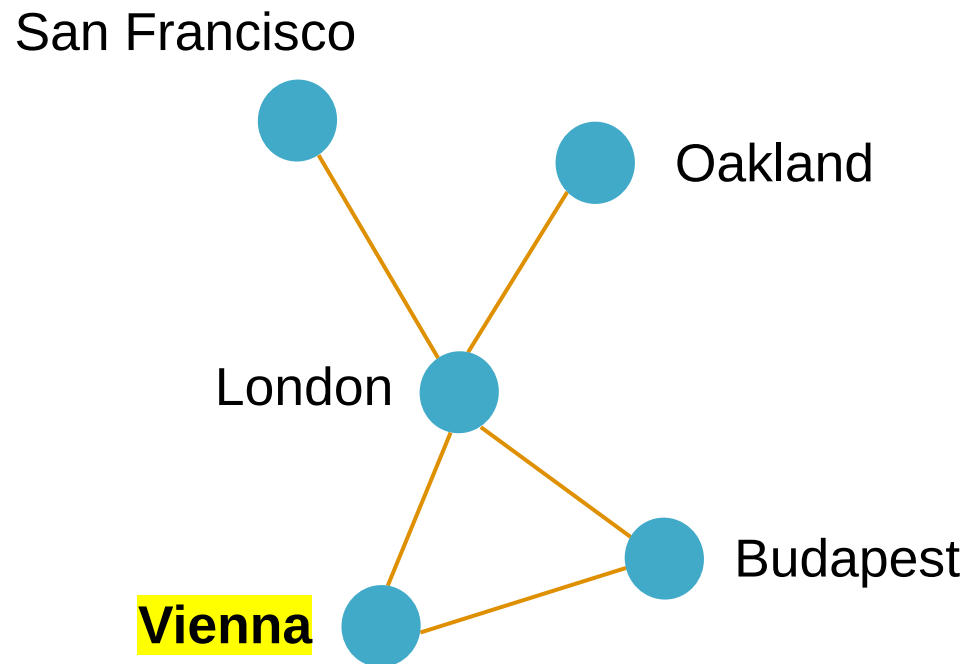


flight

Origin	Destination	Airline
VIE	LHR	BA
LHR	SFO	BA
LGW	OAK	DI
BUD	VIE	OS

airport

IATA	City
VIE	Vienna
LHR	London
LGW	London
BUD	Budapest
OAK	Oakland
SFO	San Francisco

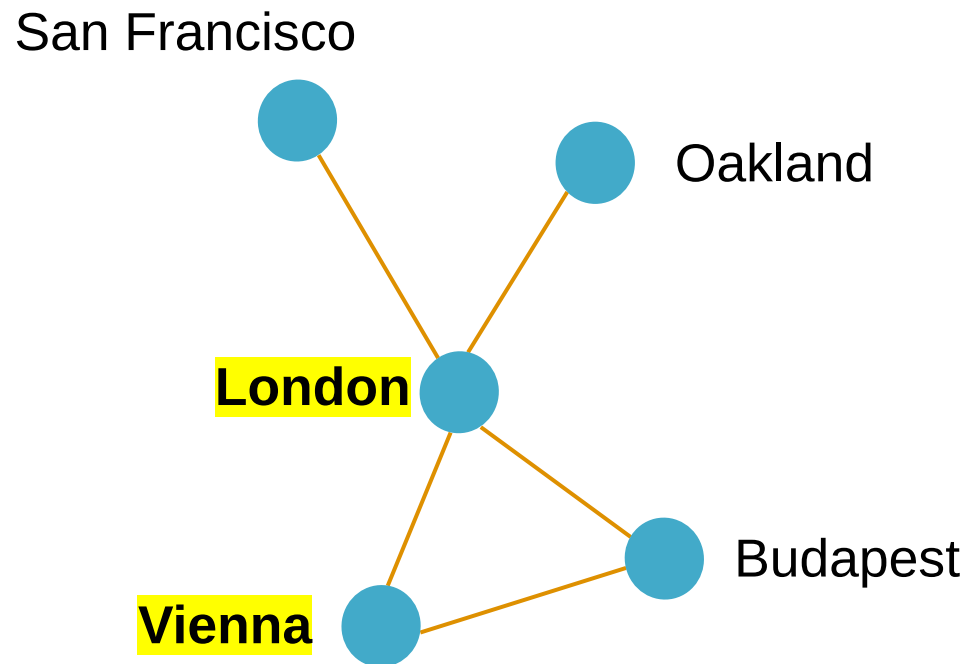


flight

Origin	Destination	Airline
VIE	LHR	BA
LHR	SFO	BA
LGW	OAK	DI
BUD	VIE	OS

airport

IATA	City
VIE	Vienna
LHR	London
LGW	London
BUD	Budapest
OAK	Oakland
SFO	San Francisco

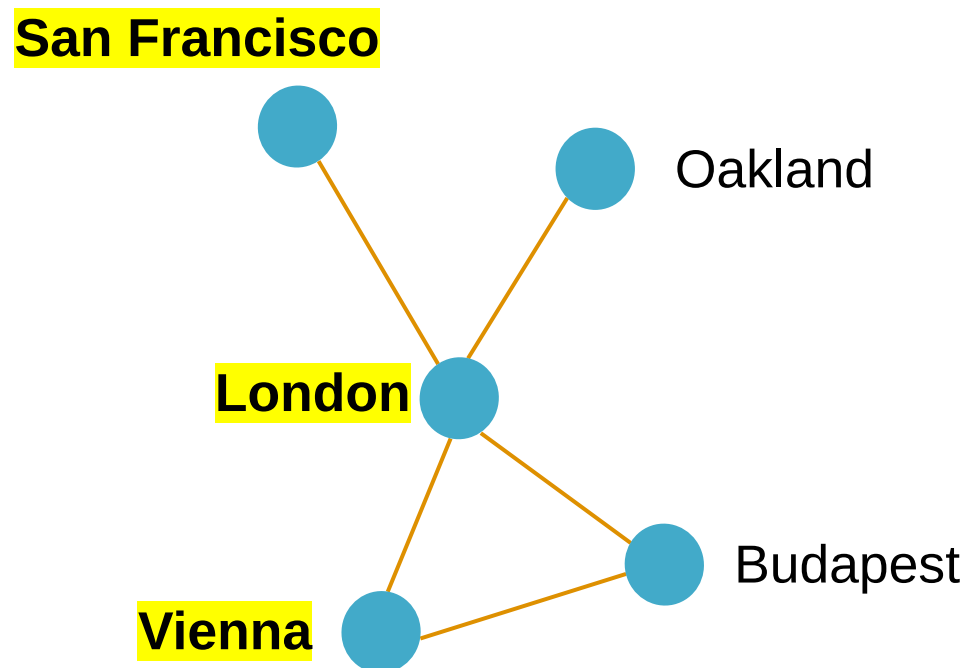


flight

Origin	Destination	Airline
VIE	LHR	BA
LHR	SFO	BA
LGW	OAK	DI
BUD	VIE	OS

airport

IATA	City
VIE	Vienna
LHR	London
LGW	London
BUD	Budapest
OAK	Oakland
SFO	San Francisco



flight

Origin	Destination	Airline
VIE	LHR	BA
LHR	SFO	BA
LGW	OAK	DI
BUD	VIE	OS

airport

IATA	City
VIE	Vienna
LHR	London
LGW	London
BUD	Budapest
OAK	Oakland
SFO	San Francisco

```
WITH RECURSIVE tconnection AS (  
    SELECT * FROM flight  
    UNION ALL  
    SELECT t.Origin, f.Destination  
    FROM tconnection t, flight f  
    WHERE t.Destination = f.Origin  
)  
SELECT * INTO connection FROM tconnection
```

SQL: Common Table Expressions

```
WITH <query_name> [ (<column> [, ...]) ] AS (  
    <SFW> )  
<main-SFW>
```

- **WITH** allows naming and defining subqueries for later use
 - called “common table expressions” (CTE)
 - similar to functions or methods in other programming languages
 - for readability and code reuse purposes; may also help optimiser
 - similar to view, but only available for current query

SQL: Common Table Expressions

STUDENTS			
SID	FIRST	LAST	EMAIL
101	Ann	Smith	...
102	Michael	Jones	(null)
103	Richard	Turner	...
104	Maria	Brown	...

EXERCISES			
CAT	ENO	TOPIC	MAXPT
H	1	Rel. Alg.	10
H	2	SQL	10
M	1	SQL	14

RESULTS			
SID	CAT	ENO	POINTS
101	H	1	10
101	H	2	8
101	M	1	12
102	H	1	9
102	H	2	9
102	M	1	10
103	H	1	5
103	M	1	7

```

WITH hw_totals AS (
    SELECT
        FROM
        WHERE
        GROUP BY
        R.sid, SUM(R.points) AS total
        Results R
        R.cat='H'
        R.sid)

SELECT
    S.first, S.last, H.total
FROM
    Students S, hw_totals H
WHERE
    S.sid = H.sid
AND
    H.total = (
        SELECT MAX(B.total)
        FROM hw_totals B );
  
```

SQL: Recursion

```
WITH RECURSIVE <query_name> [ (<column> [, ...]) ] AS (  
    <non-recursive SFW>  
    UNION ALL  
    <recursive SFW> )  
<main-SFW>
```

- Common table expressions may be recursive
 - **WITH RECURSIVE** required to indicate a recursive expression
 - has a non-recursive and a recursive part connected with **UNION** [**ALL**]
 - <query_name> may only appear in <recursive SFW>

SQL: Recursion

```
WITH RECURSIVE t(n) AS (  
    VALUES (1)  
    UNION ALL  
    SELECT n+1 FROM t WHERE n < 100  
)  
SELECT SUM(n) FROM t;
```

- Semantic: fix point ($Q(I)$ result of query Q on instance I)
 - let $R = Q_{\text{non-rec}}(I)$ be the result table, $T = Q_{\text{non-rec}}(I)$ a temporary table
 - as long as ($T \neq \emptyset$)
 - $T = Q_{\text{rec}}(I \cup (\text{query} - \text{name}, T))$ and $R = R \cup T$
 - return R
 - duplicate elimination, if **UNION** rather than **UNION ALL**

```
WITH RECURSIVE tconnection AS (  
    SELECT * FROM flight  
    UNION ALL  
    SELECT t.Origin, f.Destination  
    FROM tconnection t, flight f  
    WHERE t.Destination = f.Origin  
)  
SELECT * INTO connection FROM tconnection
```

```
SELECT * INTO connection FROM flight
```

```
LOOP
```

```
    SELECT t.Origin, f.Destination INTO connection  
    FROM connection t, flight f  
    WHERE t.Destination = f.Origin
```

```
    EXIT WHEN ...
```

```
END LOOP;
```

```
SELECT * INTO connection FROM flight
```

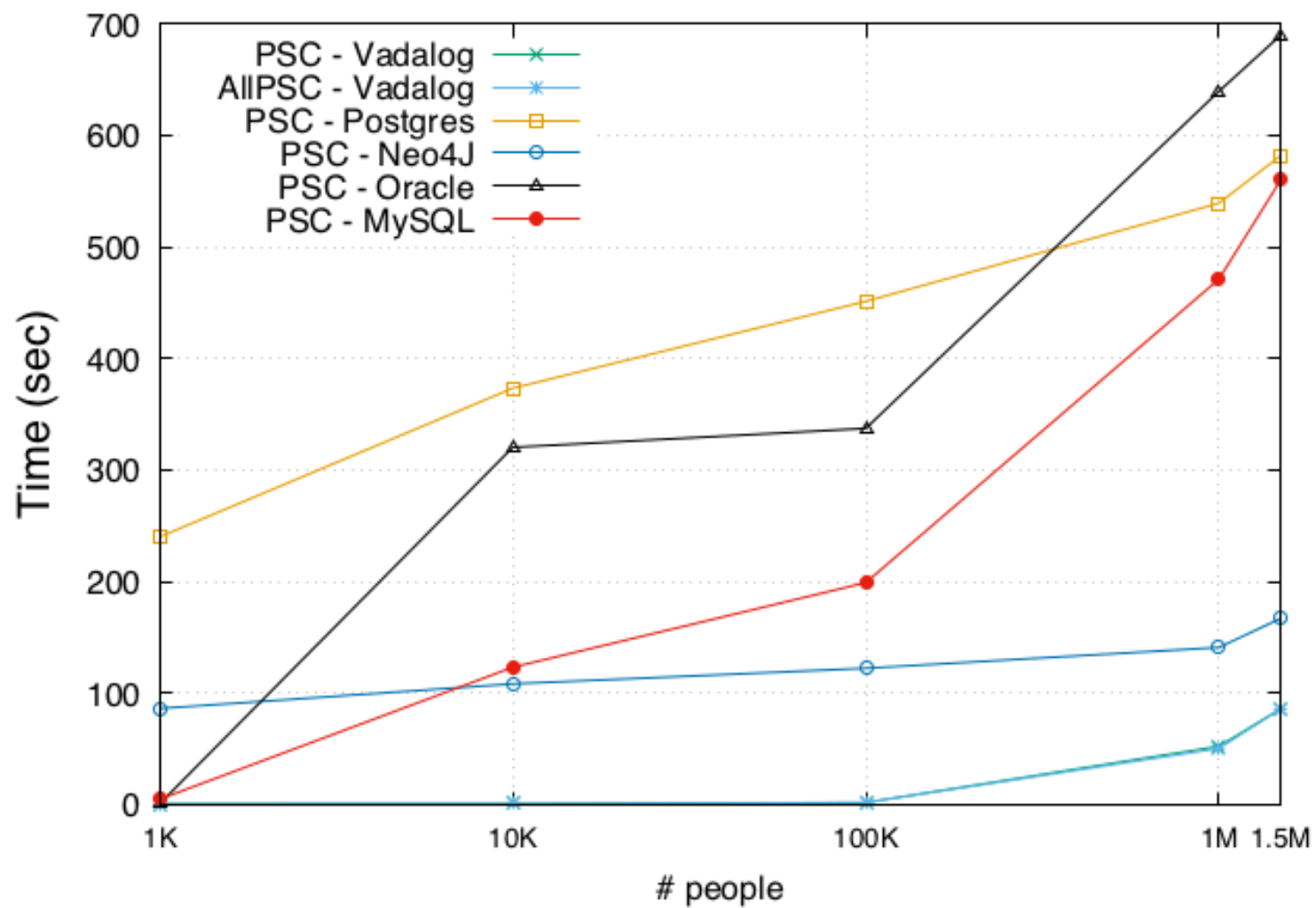
ITERATE

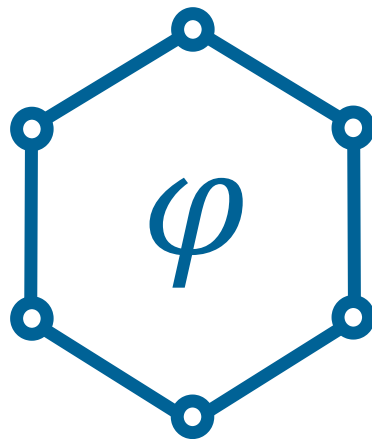
```
SELECT t.Origin, f.Destination INTO connection  
FROM connection t, flight f  
WHERE t.Destination = f.Origin
```

Recursive Query Evaluation

1. Evaluate the **non-recursive term**. For UNION (but not UNION ALL), discard duplicate rows. Include all remaining rows in the result of the recursive query, and also place them in a temporary working table.
2. So long as the working table is not empty, repeat these steps:
 - Evaluate the **recursive term**, substituting the current contents of the working table for the recursive self-reference. For UNION (but not UNION ALL), discard duplicate rows and rows that duplicate any previous result row. Include all remaining rows in the result of the recursive query, and also place them in a temporary intermediate table.
 - Replace the contents of the working table with the contents of the intermediate table, then empty the intermediate table.

(c) DBpedia PSC





Logical Knowledge in KGs

Recursion in SQL

Primer

Emanuel Sallinger